

- ① REGEX that accepts strings formatted as titles: seq. of words separated by spaces, each word being capitalized.

$^([A-Z][a-z]^*)([A-Z][a-z]^*)^*\$$

Annotations:
 - $[A-Z]$: first letter uppercase
 - $[a-z]^*$: 0 or more lowercase letters
 - $([A-Z][a-z]^*)^*$: 0 or more words after that.

- ② 4 GREP cmd. that display lines that contain words that begin with uppercase / lowercase; (file has words separated by spaces)

1) $\text{grep -E ' (^|) [aeiouAEIOU] [a-zA-Z]^* ($|) ' filename}$

Annotations:
 - $(^|)$: start of line OR space
 - $[aeiouAEIOU]$: first is vowel
 - $[a-zA-Z]^*$: anything else after
 - $($|)$: end of line or space.

2) $\text{grep -E '\<[aeiouAEIOU][a-zA-Z]^*\>' filename}$

Annotations:
 - $\<$: start of word
 - $\>$: end of word

3) $\text{grep -E -i ' (^|) [aeiouAEIOU] [a-z]^* ($|) ' filename}$

Annotations:
 - $-i$: ignore case

4) $\text{grep -Ei '\<[aeiou][a-z]^*\>' filename}$

Annotations:
 - $-Ei$: ignore case

- ③ 2 SED commands → lines that don't contain any digits (file has letters / digits sep. by spaces).

1) $\text{sed -m '/[0-9]/!p' filename}$

Annotations:
 - $-m$: nothing gets printed unless specified with p
 - $/[0-9]/$: search digits
 - $!p$: print lines that DON'T match the previous condition.

2) $\text{sed -m '/^[^0-9]^*$/p' filename}$

Annotations:
 - $^$: start of line
 - $[^0-9]^*$: 0 or more non-digits
 - $$$: end of line
 - $/p$: print lines that DO match the previous condition.

④ Cmd. line in which ~~A and B~~ C is executed $\Rightarrow$ A, B successful.

A && B && C
↓
execute if previous succeeded
execute if previous (both A and B) succeeded.

⑤ "a.txt" has 3 file names on each line.
Shell script that concatenates the first 2 files into the third one (for each line)
 $\Rightarrow$ third file exists already but the first 2.

a.txt
file1 file2 file3
file4 file5 file6
file7 file8 file9

#!/bin/bash

while read f1 f2 f3; do

~~do~~ if [-f "\$f1"] && [-f "\$f2"] && [! -e "\$f3"]; then

cat "\$f1" "\$f2" > "\$f3"

fi

concatenate $\rightarrow$ write concatenation into the third file.

done < a.txt

$\rightarrow$ read all lines from a.txt.

⑥ Add code s.t. line-count (variable) holds the nr. of lines from the stdout of the ls -la /etc command.

#include <stdio.h>

#include <stdlib.h> // for popen

//...

int line_count;

FILE *f = popen("ls -la /etc", "r"); $\rightarrow$ run that command

char line[1024];

line_count = 0;

while (fgets(line, sizeof(line), f)) $\rightarrow$ read one line

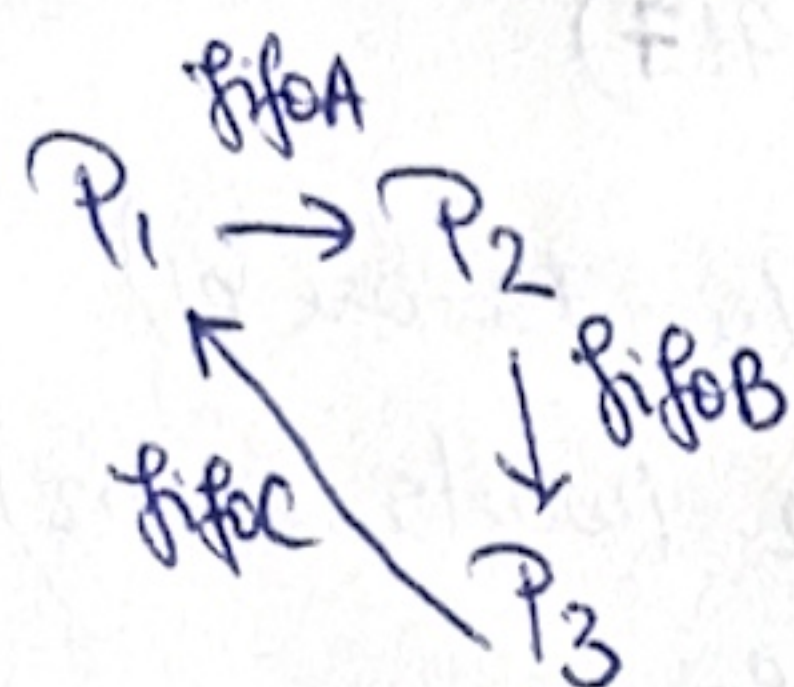
line_count++;

pclose(f); $\rightarrow$ close pipe

//...

printf("Total lines: %d\n", line_count);

P1, P2, P3 need to communicate in a circular fashion. 



mkfifo commands?
open() calls?

mkfifo FIFO_A
mkfifo FIFO_B
mkfifo FIFO_C

P1:

```

mkfifo("FIFO_A", 0600);
mkfifo("FIFO_C", 0600);
int FIFO2 = open("FIFO_A", O_WRONLY);
int FIFO31 = open("FIFO_A", O_RDONLY);
  
```

P2:

```

mkfifo("FIFO_A", 0600);
mkfifo("FIFO_B", 0600);
int FIFO23 = open("FIFO_B", O_WRONLY);
int FIFO12 = open("FIFO_B", O_RDONLY);
  
```

P3:

```

mkfifo("FIFO_C", 0600);
mkfifo("FIFO_B", 0600);
int FIFO31 = open("FIFO_C", O_WRONLY);
int FIFO23 = open("FIFO_B", O_RDONLY);
  
```

⑧ Protect the critical section (without mutexes)

⇒ use binary semaphore.

```

#include <pthread.h>
#include <semaphore.h>
sem_t sem; // semaphore declaration
int m = 0;
void* f(void* v) {
    sem_wait(&sem); // enter critical
    if (m < 500) { m++; }
    if (m > 600) { m = 0; }
    sem_post(&sem); // leave critical
    return NULL;
}
  
```

```

int main() {
    pthread_t thread;
    sem_init(&sem, 0, 1); // allows 1 thread (binary)
    pthread_create(&t, NULL, f, NULL);
    pthread_join(t, NULL);
    sem_destroy(&sem); // cleanup
    return 0;
}
  
```

address of semaphore s.

⑨ $T, N_1, N_2, N_3 > 0$ s.t. program is ok. (see 12 jun 913)

! each loop $\rightarrow$ program creates: 1 thread for f_1 (waits on b_1)
2 threads for f_2 (waits on b_2)
3 threads for f_3

pthread_barrier_t b_1, b_2, b_3 ;

void * f_1 (void * a) { wait(& b_1) } $\leftarrow$ type 1 threads wait at b_1

-- * f_2 -- a

-- * f_3 -- a

----- & b_2 -----
----- & b_3 -----

int main() {

int T, N_1, N_2, N_3 ;

pthread_t all_threads[6*T]; $\leftarrow$ 6*T threads, linear array.

pthread_barrier_init(& b_1 , NULL, N_1); $\leftarrow$ b_1 opens when N_1 threads arrive.

- init(& b_2 , NULL, N_2);

- init(& b_3 , NULL, N_3);

for(... T ...) { $\leftarrow$ repeat T times

int base_idx = $i * 6$;

pthread_create(&all_threads[base_idx + 0], NULL, f_1 , NULL);

two f_2 threads { [... +1], NULL, f_2 , ...
[... +2], NULL, f_2 , ...

three f_3 threads { [... +3], NULL, f_3 , ...
[... +4], NULL, f_3 , ...
[... +5], NULL, f_3 , ...

}
for(... $6 * T$...)

$\leftarrow$ wait for all threads

... join ...

}

... destroy(& b_1);

(& b_2);

(& b_3);

return 0;

f_1 threads $\rightarrow T$: $T \% N_1 = 0$

f_2 threads $\rightarrow 2T$: $2T \% N_2 = 0$

f_3 threads $\rightarrow 3T$: $3T \% N_3 = 0$.

$\Rightarrow \alpha: T = 4$,

$N_1 = 4$

$N_2 = 2$

$N_3 = 3$

$T = 5, N_1 = 5, N_2 = 5, N_3 = 5$

• f_1 threads = $T = 5$; barrier $b_1 = 5 \Rightarrow$ releases once.

• f_2 threads = $2T = 10$; $b_2 = 5 \Rightarrow$ releases twice

• f_3 threads = $3T = 15$; $b_3 = 5 \Rightarrow$ releases 3 times.

MCFQ scheduling policy

= Multi-level-Feedback queue.

- good CPU utilization
- avoids starvation
- good response time

- processes are placed in multiple priority queues.
- new process ⇒ highest-priority queue ✓
- uses entire quantum ⇒ lower priority. ✓
- before quantum expires ⇒ block ⇒ stay in the same queue.
- scheduler chooses from highest non-empty queue.
- processes are boosted back up periodically (prevent starvation)

STAY NEAR TOP: - when ∃ interactive / I-O operations. (processes)

MOVE DOWN: - when ∃ CPU-bound ~~poli~~ processes
(PRIORITY)

Q11 What happens to a symbolic link when the file to which it points to gets deleted?

⇒ Symbolic link becomes dangling link because its target no longer exists.

- points to the path of the target file
- can span across ≠ file systems
- invalid if target file is deleted

• ln -s

ex: > ln -s a.txt link } ⇒ link → a.txt (still exists)
run a.txt } ∄ a.txt.